@prism project

Christophe BAL

1 Jan 2026 – Version 1.3.0

The `@prism` project [1] provides small size color palettes that can be used to create expressive color maps for graphics in different contexts.

Last changes

1.3.0
2026-01-01

Break.

- `luadraw` product becomes `Lua` product.
- `Lua` product: the dict-like variable `palNames` has been removed. Therefore, the responsibility of adding this specific variable will lie with `luadraw`.

New.

- New `CSS` and LaTeX products have been coded.
- The `Palettable` palettes have been added.
- Three semantic palettes `TdocBW`, `TdocCol` and `TdocDark` have been created to be used by the `tutodoc` class (LaTeX creation process used).
- Added an `online demo of @prism` (the `CSS` product showcase is used).

Update.

- Manual.
  - Added lists of renamed and removed palettes.
  - Added two appendixes, one with all palettes grouped by user-friendly categories, and another showing similar palettes.
  - Added explanations about the utility of palettes for non-designers.
  - Added explanations about the minimal common optional sets of extra features implemented.
- Better palette references.
- Better showcases.
  - Several category `PDF` files are proposed (one `PDF` per category and theme).
  - 2D chart and Delaunay triangulation added.

Fix.

- `Lua` product: the `use-std.tex` and `use-dark.tex` codes have been fixed.

[1] The name comes from "***@ · esthetic P · roducts*** *for* ***R · epresenting I · nformative S · cientific M · aps***". This name is a double play on words: [1] a prism splits light into an informative spectrum, symbolizing how data are decomposed into meaningful color, and [2] "@" read as "at" indicates where the light meets the prism to be broken down into an informative spectrum.

Contents

I. Motivations **3**

II. Where do the color palettes come from? **4**

III. Reuse from... **5**

IV. How to choose a palette? **6**

V. Supported implementations **6**
1. JSON, the versatile default format 6
2. CSS 7
3. LaTeX 8
   a. Basic use 8
   b. Creating palettes from scratch 8
   c. Creating palettes from existing ones 9
   d. Retrieving the internal definition of a palette 9
4. Lua 9
   a. Basic use 9
   b. Creating palettes from existing ones 10

VI. Contribute via Git **10**
1. Complete the translations 10
   a. The en folder 10
   b. The changes folder 10
   c. The status folder 10
   d. The README.md and LICENCE.txt files 11
   e. New translations 11
2. Improving the source code 11

VII. History **12**

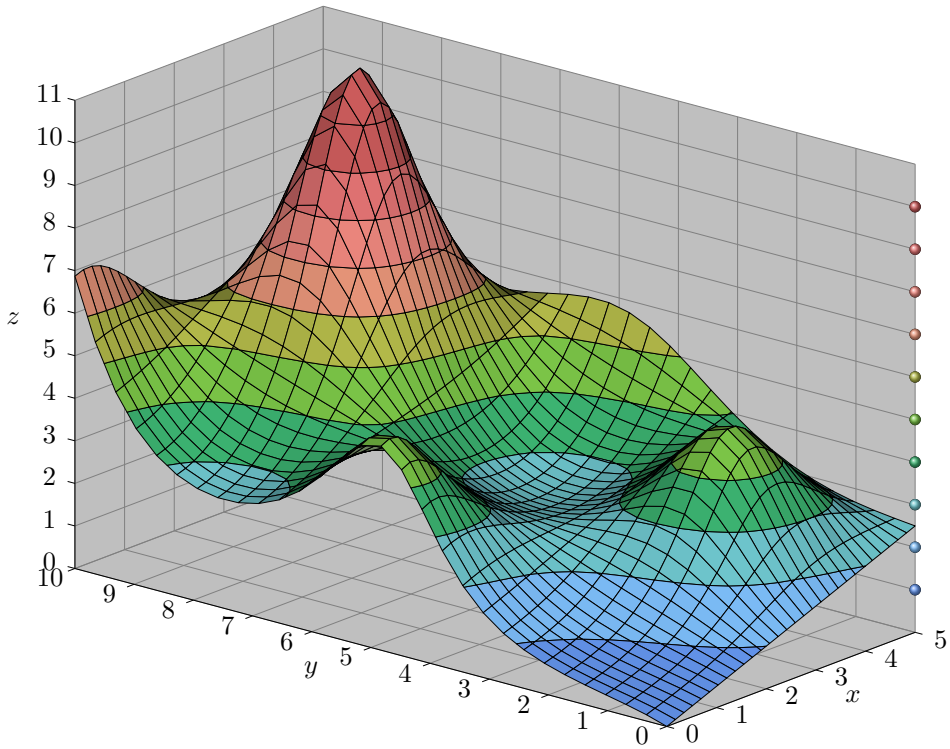
Appendix 1 – The 261 palettes at a glance **14**
- Colorblind-friendly palettes – 40 palettes 14
- Semantic palettes – 37 palettes 16
- Two-color palettes – 79 palettes 18
- Three-color palettes – 93 palettes 22
- Four-color palettes – 33 palettes 27
- More color palettes – 19 palettes 29

Appendix 2 – Similar palettes **30**

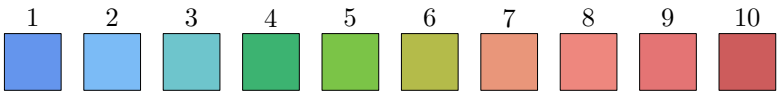
I. Motivations

This project was born out of a desire to enhance `luadraw` with a set of color palettes to easily produce graphics like the 3D plot shown below. Despite the availability of palette generation tools such as `Coolors`, the choice has been made to offer pre-built palettes, because this serves several key purposes for non-designers.

- **Reduced cognitive load:** pre-configuration eliminates design decisions, allowing to focus on content.
- **Consistency:** it becomes easy to ensure visual coherence across documents and projects, regardless of technology.
- **Accessibility compliance:** some pre-configured palettes guarantee colorblind-safe combinations.
- **Perceptual uniformity:** most palettes ensure distinguishable and meaningful colors for data visualization, while others serve artistic purposes.



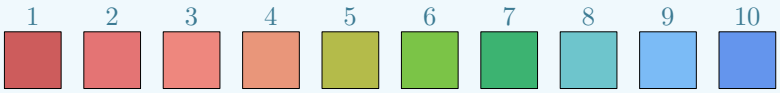
Technically, to generate the graphics in this section, `luadraw` is provided with a finite list of colors and uses linear interpolations to calculate the intermediate color values. In the example above, the finite color palette used is defined as follows.



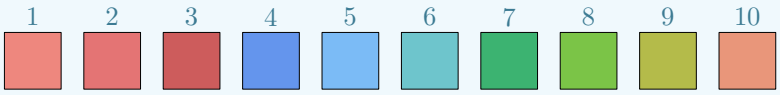
Using this palette, `luadraw` can produce the following spectrum, allowing us to create the graphic above.

Note.

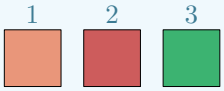
Using the `Lua` implementation of `@prism`, see the section V-4, via `luadraw`, we can create the palettes below made from the previous one named `'GeoRainbow'`. Each instruction used is given below each palette.




```
getPal('GeoRainbow', {reverse = true})
```

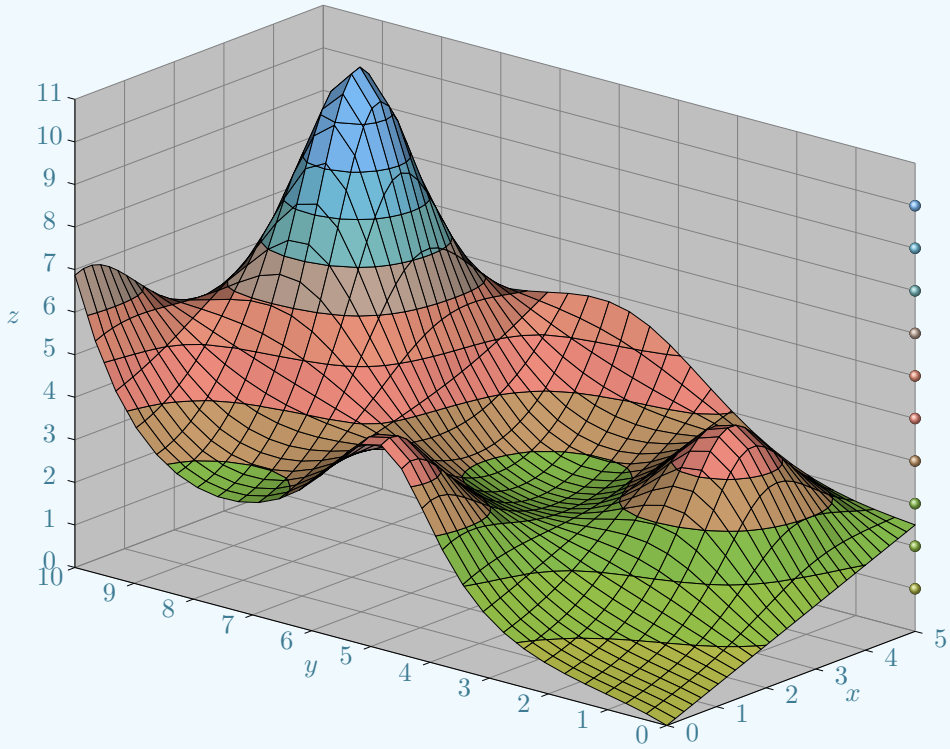


`getPal('GeoRainbow', {shift = 3})`



`getPal('GeoRainbow', {extract = {7, 10, 4}})`

This features provide remarkable creative flexibility: with the same surface as before, using the setting `getPal('GeoRainbow', {extract = {2, 3, 7, 8, 5, 6}, reverse = true})` *instead of* `getPal('GeoRainbow')`*, we change the visual tone, shifting from a seaside feel to a snow-covered world.*



II. Where do the color palettes come from?

`@prism` includes some original creations, but most color palettes are derived from the projects listed below by segmenting their color maps into 10-value palettes.

- `Asymptote` is used, but currently offers nothing beyond `Matplotlib` (despite different implementations).
- `CartoColor` palettes are extracted from `Palettable` project.
- `cmocean` palettes are extracted from `Palettable` project.
- `Colorbrewer` provides professional color palettes for mapping and data visualization.
- `Light and Bartlein` palettes are extracted from `Palettable` project.
- `Matplotlib` compiles color maps from diverse projects, serving as the foundation for the initial palette list.
- `MyCarta` palettes are extracted from `Palettable` project.
- `Plotly` palettes are extracted from `Palettable` project.
- `Scientific Coulour Maps` provides palettes designed for colorblind accessibility.
- `Tableau` palettes are extracted from `Palettable` project.
- `Wes Anderson Palettes` palettes are extracted from `Palettable` project.

We retain only palettes that comply with the following rules.

- **No repetition.** Unlike `Matplotlib`,[2] `@prism` use a one-to-one map from names to palettes.
- **No reversed versions.** Unlike `Matplotlib`,[3] `@prism` never includes reversed palettes as fixed data.

Note.

Adding new palettes to `@prism` is straightforward (no coding skills required). See section VI-2 to get started.

III. Reuse from...

Here are the key points to remember when using palettes similar to those offered by projects listed in the section II.

1. `@prism` uses strict alphanumeric `CamelCase` naming. Therefore, palette names such as `berlin` and `gist_heat` become `Berlin` and `GistHeat` respectively.
2. `Matplotlib` palettes with a name ending with `_r` (reversed color order) are not included in `@prism`.
3. The following presents palettes from projects other than `Matplotlib` that have been kept but renamed: $\boxed{\Rightarrow}$ indicates a name modification, with the `@prism` name displayed on the right.

CartoColors	`Prism`	$\Rightarrow$	`PrismCC`
Plotly	`Bluered`	$\Rightarrow$	`BlueRedPly`
	`Rainbow`	$\Rightarrow$	`RainbowPly`
Tableau	`Gray`	$\Rightarrow$	`GrayTab`

4. The following palettes are excluded because they duplicate `@prism` palettes either directly or in reversed order, except that exact duplicates (same name and colors) are omitted when they don't come from `Matplotlib`, and we use $\boxed{=}$ for equality, $\boxed{\rightleftharpoons}$ for reversal, and the rightmost palettes are the ones retained in `@prism`.

Cubehelix	`Classic`	$=$	`Cubehelix`
Matplotlib	`GistGray`	$\rightleftharpoons$	`Binary`
	`GistGrey`	$\rightleftharpoons$	`Binary`
	`GistYarg`	$=$	`Binary`
	`GistYerg`	$=$	`Binary`
	`Gray`	$\rightleftharpoons$	`Binary`
	`Grey`	$\rightleftharpoons$	`Binary`
	`Greys`	$=$	`Grays`
Plotly	`D3`	$=$	`Tab10`
cmocean	`Balance`	$=$	`Vik`
	`Gray`	$=$	`Binary`

Caution.

Most `@prism` implementations add the `pal` prefix to standardized `CamelCase` names. See the section V.

Note.

Most `@prism` implementations provide methods to easily obtain reversed palettes, sub-palettes, and color-shifted palettes. See the section V.

[2] Some `Matplotlib` palettes are duplicated, likely for historical reasons.
[3] Most `Matplotlib` color maps have a reversed version named with the `_r` suffix, possibly for performance reasons.

IV. How to choose a palette?

Here are a few tips to help you select your color palette.

1. An `online demo of @prism` proposes an intuitive way to look for a palettes.
2. Appendix 1 page 14 presents all palettes, organized by category, along with a visualization of their continuous spectra and their corresponding discrete palette definitions.
3. The categories listed in appendix 1 are also demonstrated through use cases in the `showcase` folder, available in light and dark themes.
4. Appendix 2 page 30 groups visually similar palettes together.

V. Supported implementations

All implementations are located in the `products` folder. Each implementation provides palette definitions and supports the use of a single color. When available, the following actions can be performed to create new palettes:

- Select specific colors from an existing palette using their indices.
- Shift the palette left (negative value) or right (positive value) by any number of steps.
- Reverse the order of the colors.

Important.

To explain how new palettes can be built, we will refer to the colors in the standard palette as `coul_1`, `coul_2`, etc., and suppose that the extracted indices are `{1, 3, 6, 9}`, the shift used is 1+, and the `reverse` option is enabled. The new palette will then be built sequentially as follows: first `{coul_1, coul_3, coul_6, coul_9}` (extraction), second `{coul_9, coul_1, coul_3, coul_6}` (shift to the right), and finally `{coul_6, coul_3, coul_1, coul_9}` (reverse).

Note.

Extra features are limited to discrete palette operations. For example, color interpolation is not provided, as this is usually handled out of the box by visualization and formatting tools.

1. JSON, the versatile default format

A `palettes.json` file containing only palette definitions is provided by default, allowing unsupported programming languages to integrate `@prism` palettes. Below are the first lines of the file.

```
{
  "Accent": [
    [0.498039, 0.788235, 0.498039],
    [0.690196, 0.705881, 0.757298],
    [0.882352, 0.721568, 0.661437],
    [0.99477, 0.835294, 0.550326],
    [0.913289, 0.935947, 0.610021],
    [0.306317, 0.487581, 0.680174],
    [0.700653, 0.146404, 0.562091],
    [0.855772, 0.162962, 0.316775],
    [0.671459, 0.366448, 0.159041],
    [0.4, 0.4, 0.4]
  ],
  ...
}
```

2. CSS

> **Note.**
>
> *The HTML, CSS, and JavaScript files for product development and demonstration were created using Claude and Gemini AI assistants.*

The very basic CSS implementation provides only ten variables per palette, named according to the pattern `--pal<name>-<nb>`, where `<name>` is the standard palette name and `<nb>` is the desired index ranging from 1 to 10. Each palette color variable is defined as an RGB value using percentage notation. For example, the `GistHeat` palette color definitions look like the following partial code.

```
:root {
  /* Previous palettes. */

  --palGistHeat-1 = rgb(0% 0% 0.0%)
  --palGistHeat-2 = rgb(10.5882% 0% 0%)
  --palGistHeat-3 = rgb(21.1764% 0% 0%)
  /* ... With 7 more RBG colors.*/

  /* Additional palettes. */
}
```

Here are two possible use cases.

```
.warning-text {
  color: var(--palGistHeat-3);
}

.gist-heat-gradient {
  background: linear-gradient(
    90deg,
    var(--palGistHeat-1),
    var(--palGistHeat-2),
    var(--palGistHeat-3),
    var(--palGistHeat-4),
    var(--palGistHeat-5),
    var(--palGistHeat-6),
    var(--palGistHeat-7),
    var(--palGistHeat-8),
    var(--palGistHeat-9),
    var(--palGistHeat-10)
  );
}
```

The example below demonstrates creating gradient variables through selective color extraction and custom reordering.

```
:root {
  --transformed-gist-heat-gradient: linear-gradient(
    90deg,
    var(--palGistHeat-6),
    var(--palGistHeat-3),
    var(--palGistHeat-9),
    var(--palGistHeat-1)
  );
}

.transformed-gist-heat-gradient {
  background: var(--transformed-gist-heat-gradient);
}
```

3. LaTeX

a. Basic use

To use a color from a palette, use `\palUse{<name>}{<index>}` where `<name>` is the standard palette name (without prefix), and `<index>` is the color number (ranging from 1 to 10). For example, `\palUse{GistHeat}{8}` is the eighth color of the `GistHeat` palette, an `xcolor` format color that can be easily used as shown in the following compilable example.

```
\documentclass{article}

\usepackage{palettes}
\usepackage{tikz}

\begin{document}

\textcolor{\palUse{GistHeat}{8}}{\bfseries Colored text.}

Representation of the color palette.

\begin{tikzpicture}
  \foreach \i in {1,...,10} {
    \fill[\palUse{GistHeat}{\i}]
      (1.25*\i - 1, 0) rectangle (1.25*\i, 1);
  }
\end{tikzpicture}

\end{document}
```

b. Creating palettes from scratch

For creating new palettes manually, the following high-level commands are available.

1. `\palCreateFromRGB` creates a palette by entering it as an array-like variable, while `\palCreateFromNames` works with named colors.
2. `\palSize{<name>}` returns the palette size (useful for loops, for example).

The following example demonstrates the `\palCreateFromRGB` and `\palCreateFromNames` commands (we don't have put `\usepackage{palettes}`).

```
\usepackage[svgnames]{xcolor}

\palCreateFromRGB{MyRGBPal}{
  {0.0, 0.0, 0.0},
  {0.4, 0.0, 0.2},
  {0.8, 0.2, 0.0},
}

\palCreateFromNames{MyNameUsePal}{
  YellowGreen,
  green!60!black,
  LimeGreen!80,
}
```

Note.

All built-in palettes are created using the `\palCreateFromRGB` macro.

A lower-level approach is also available through the following commands.

1. `\palNew{<name>}` initializes a new (empty) palette.
2. `\palAddNames{<name>}{<color-using-names>}` appends a color defined with named colors to the palette.

3. `\palAddRGB{<name>}{<r>, <g>, <b>}` appends an RGB color to the palette, where `<r>`, `<g>`, and `<b>` are decimal values ranging from 0 to 1.

The following example demonstrates the flexibility offered by these low-level commands.

```
\usepackage[svgnames]{xcolor}

\palNew{LowLevelPal}
\palAddNames{LowLevelPal}{IndianRed}
\palAddRGB{LowLevelPal}{0.0, 0.0, 0.0}
\palAddNames{LowLevelPal}{green!60!black}
\palAddRGB{LowLevelPal}{0.8, 0.2, 0.0}
```

c. Creating palettes from existing ones

Building new palettes by transforming existing ones can be achieved using the `\palCreateFromPal` command, which has the signature `\palCreateFromPal{<new-name>}[<options>]{<existing-name>}`. The following example shows how to do this (all options are used).

```
\palCreateFromPal{BlackbodyTransformed}[
  extract = {1, 3, 6, 9},
  shift   = 1,
  reverse
]{Blackbody}
```

Tip.

`\palCreateFromPal{<new-name>}{<existing-name>}` build a copy of an existing palette.

d. Retrieving the internal definition of a palette

The internally stored definition of a palette named `MyPal`, for example, is `\g_palette_MyPal_seq` which is a LaTeX3 variable (keep in mind the pattern `\g_palette_PaletteName_seq`).

Note.

Variables of type `\g_palette_PaletteName_seq` are not used internally to retrieve the colors themselves; they are only there for technical reasons related to the development process of new palettes via LaTeX.

4. Lua

Note.

Initially, the `@prism` project was created to provide ready-to-use color palettes for `luadraw`, a package that greatly facilitates the creation of high-quality 2D and 3D plots using LuaLaTeX and `TikZ`. The `Lua` implementation is now integrated into `luadraw`.

a. Basic use

The `Lua` palette names all use the prefix `pal` followed by the name available in the file `palettes.json`. You can access a palette by two ways.

- `palGistHeat` is a `Lua` variable.
- `getPal('GistHeat')` and `getPal('palGistHeat')` are equal to `palGistHeat`.

The `Lua` palette variables are arrays of ten arrays of three floats (making it straightforward to use a color from a palette). For example, the definition of `palGistHeat` looks like the following partial code.

```
palGistHeat = {
    {0.0, 0.0, 0.0},
    {0.105882, 0.0, 0.0},
    {0.211764, 0.0, 0.0},
    -- ... With 7 more RBG colors.
```

```
}
```

b. Creating palettes from existing ones

The `getPal` function provides options to build new palettes by transforming existing ones. The following example shows how to do this (all options are used).

```
BlackbodyTransformed = getPal(
    'Blackbody',
    {
        extract = {2, 5, 8, 9},
        shift   = 1,
        reverse = true
    }
)
```

VI. Contribute via Git

Caution.

Never use the `main` *branch, which is for freezing the latest stable versions of all the projects in the mono repository* `https://github.com/projetmbc/for-writing`.

1. Complete the translations

Important.

Although we're going to explain how to translate the documentation, it doesn't seem relevant to do so, as English should suffice these days.

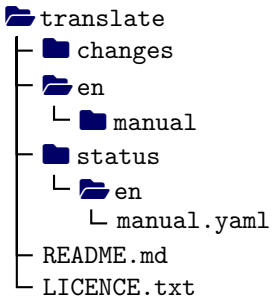


Figure 1: Simplified view of the translation folder

The translations are roughly organized as in figure 1 where just the important folders for the translations have been "*opened*".[4] **A little further down, the section VI-1-e explains how to add new translations**.

a. The en folder

This folder, managed by the author of `@prism`, contains files easy to translate even if you're not a coder.

b. The changes folder

This folder is a communication tool where important changes are indicated without dwelling on minor modifications specific to one or more translations.

c. The status folder

This folder is used to keep track of translations from the project's point of view. Everything is done via well-commented `YAML` files, readable by a non-coder.

[4] This was the organization on October 26, 2025.

d. The README.md and LICENCE.txt files

The `LICENCE.txt` file is aptly named, while the `README.md` file takes up in English the important points of what is said in this section about new translations.

e. New translations

Note.

The folder `manual` *is reserved for documentation. It contains* `TEX` *files that can be compiled directly for real-time validation of translations.*

Warning.

Only start from the `en` *folder, as it's the responsibility of the* `@prism` *author.*

Let's say you want to add support for Italian.[5] To do this, you must use `Git` as follows.

1. Via `https://github.com/projetmbc/for-writing/tree/aprism/@prism`, recover the entire project folder. Do not use the `main` branch, which is used to freeze the latest stable versions of all the projects in the mono repository `https://github.com/projetmbc/for-writing`.
2. In the `@prism/contrib/translate` folder, create an `it` copy of the `en` folder, where `it` is the short name of the language documented in the page "*IIETF language tag*" from `Wikipedia`.
3. Once the translation is complete in the `it` folder, share it via `https://github.com/projetmbc/for-writing/tree/aprism/@prism` using a classic `git push`.

2. Improving the source code

Participation as a coder is made via the repository `https://github.com/projetmbc/for-writing/tree/aprism/@prism` corresponding to the `@prism` development branch. Here is what you can do, details can be found in the file `https://github.com/projetmbc/for-writing/blob/aprism/@prism/contrib/products/README.md`.

1. Create new palettes within an existing implementation. No coding skills required.
2. Propose a new implementation in your favorite programming language.
3. Combine both approaches.

[5] As mentioned above, there is no real need for the `doc` folder.

VII. History

1.3.0
2026-01-01

Break.

- luadraw product becomes Lua product.
- Lua product: the dict-like variable palNames has been removed. Therefore, the responsibility of adding this specific variable will lie with luadraw.

New.

- New CSS and LaTeX products have been coded.
- The Palettable palettes have been added.
- Three semantic palettes TdocBW, TdocCol and TdocDark have been created to be used by the tutodoc class (LaTeX creation process used).
- Added an online demo of @prism (the CSS product showcase is used).

Update.

- Manual.
  - Added lists of renamed and removed palettes.
  - Added two appendixes, one with all palettes grouped by user-friendly categories, and another showing similar palettes.
  - Added explanations about the utility of palettes for non-designers.
  - Added explanations about the minimal common optional sets of extra features implemented.
- Better palette references.
- Better showcases.
  - Several category PDF files are proposed (one PDF per category and theme).
  - 2D chart and Delaunay triangulation added.

Fix.

- Lua product: the use-std.tex and use-dark.tex codes have been fixed.

1.2.1
2025-11-09

Fix.

- Equal palettes: the floating point equality uses now a correct tolerance.

Break.

- Palettes: the extra Greys has been removed (it is equal to Grays).

New.

- Similar palettes: two PDF files show similar palettes in standard and black modes (semi-automated process used).

Update.

- luadraw product: the associative array palNames has been added for compatibility reasons with the luadraw package.
- BlindFish palette: the last color variation has been made smoother (luadraw process used).

1.2.0
2025-10-29

Break.

- Palettes: all final palettes now consist of 10 colors.
- luadraw products: the getPal dictionary array has been converted into a function accepting string palette names (with or without pal prefix). See below.

New.

- Palettes.
  - Added Lemon and ShiftRainbow palettes (luadraw creation process used).
  - Added 37 palettes from the Scientific Coulour Maps project.
- luadraw product: accessing a palette and creating new ones can be made using the getPal function which has an optional argument options (dict-like array) with the following keys and their values.
  - extract: a list of non-zero integers used to extract specific colors from the palette (the order is preserved).
  - reverse: a boolean value indicating whether to reverse the palette color order (false by default).
  - shift: an integer value for applying a circular color shift to the palette.
- Documentations
  - Added English PDF manual.
  - Showcase: two PDF files demonstrate the use of each palette (white and dark modes).

1.1.0
2025-10-14

Break.

- Duplicate palettes and those that are reverse of others are ignored (strict equalities only).

New.

- New palettes added: `BurningGrass`, `GeoRainbow` and `PastelRainbow` (`luadraw` creation process used).
- The `luadraw` palette product has a new dictionary like variable `getPal` to access a palette using its name (as a string variable).

Update.

- Palette contributions: in the mandatory `extend.py` file, the `build_code` function must work with the dictionary of all the palettes, and manage a credit to the `@prism` project.

1.0.0
2025-10-11

First public version of the project.

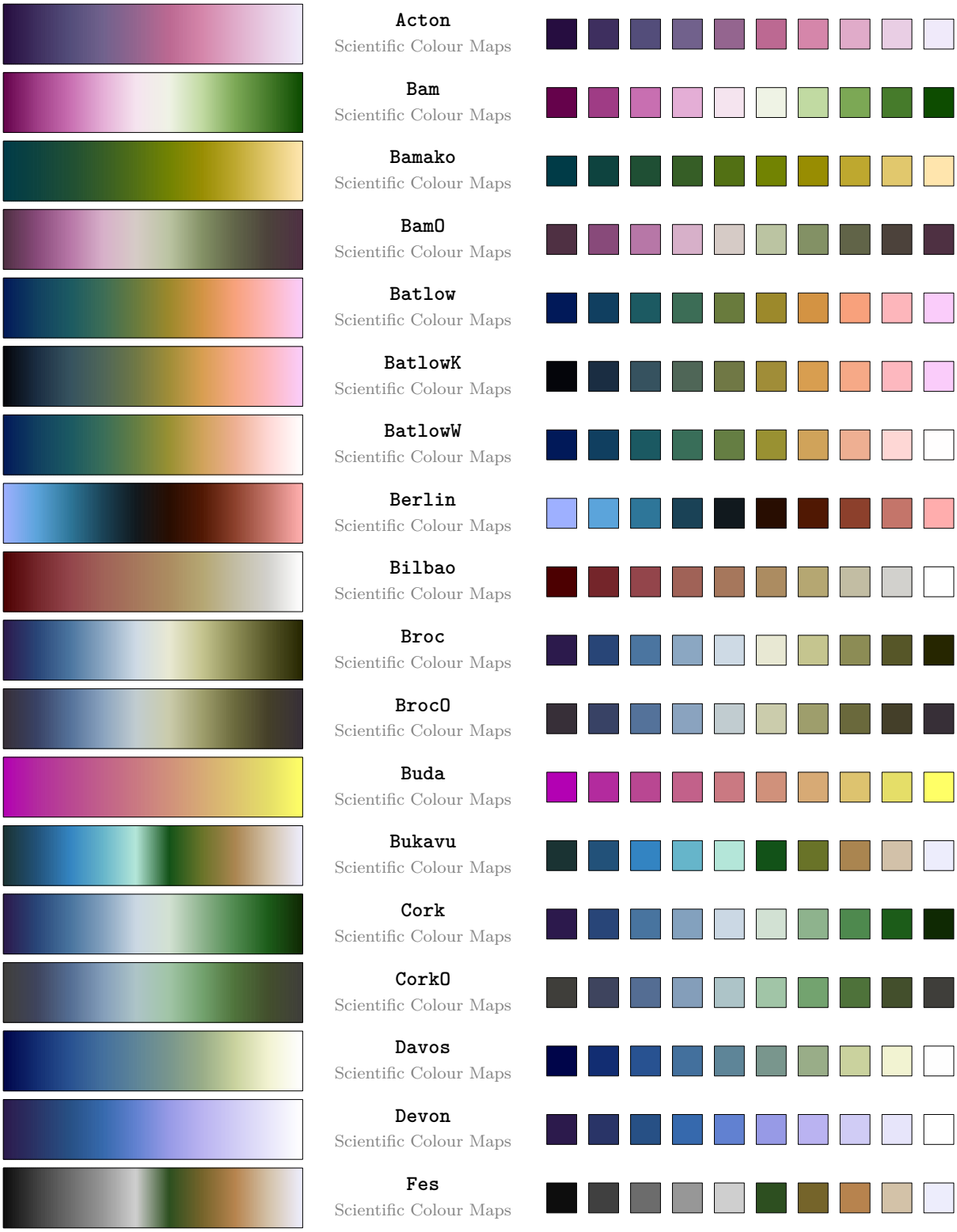
Appendix 1 – The 261 palettes at a glance

The palette names used in this appendix are standard, but most `@prism` implementations add the `pal` prefix.

Important.

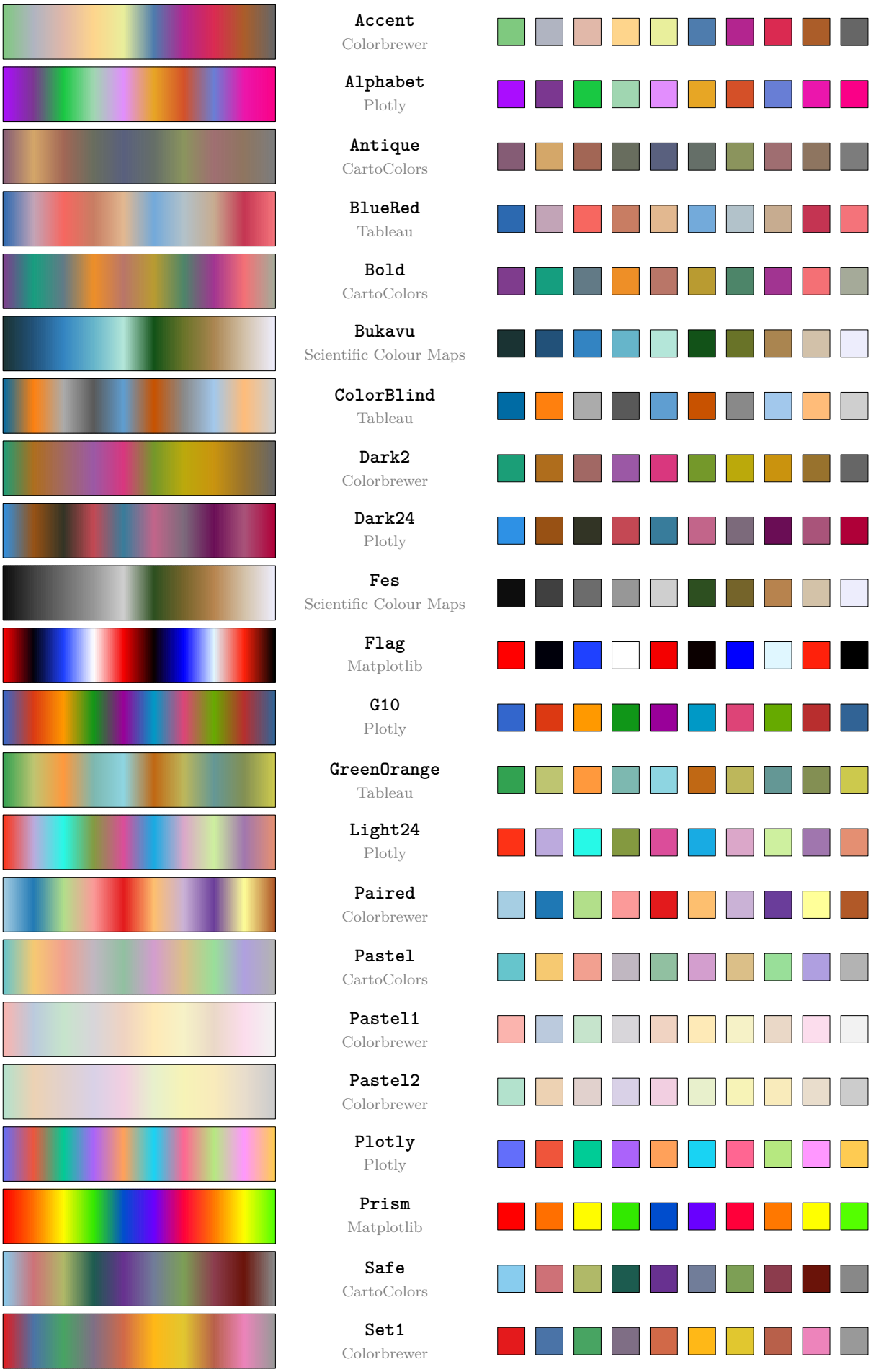
Categories were generated semi-automatically using a program, followed by manual selection to obtain relevant choices. If you identify any errors, please contact the author of `@prism`.

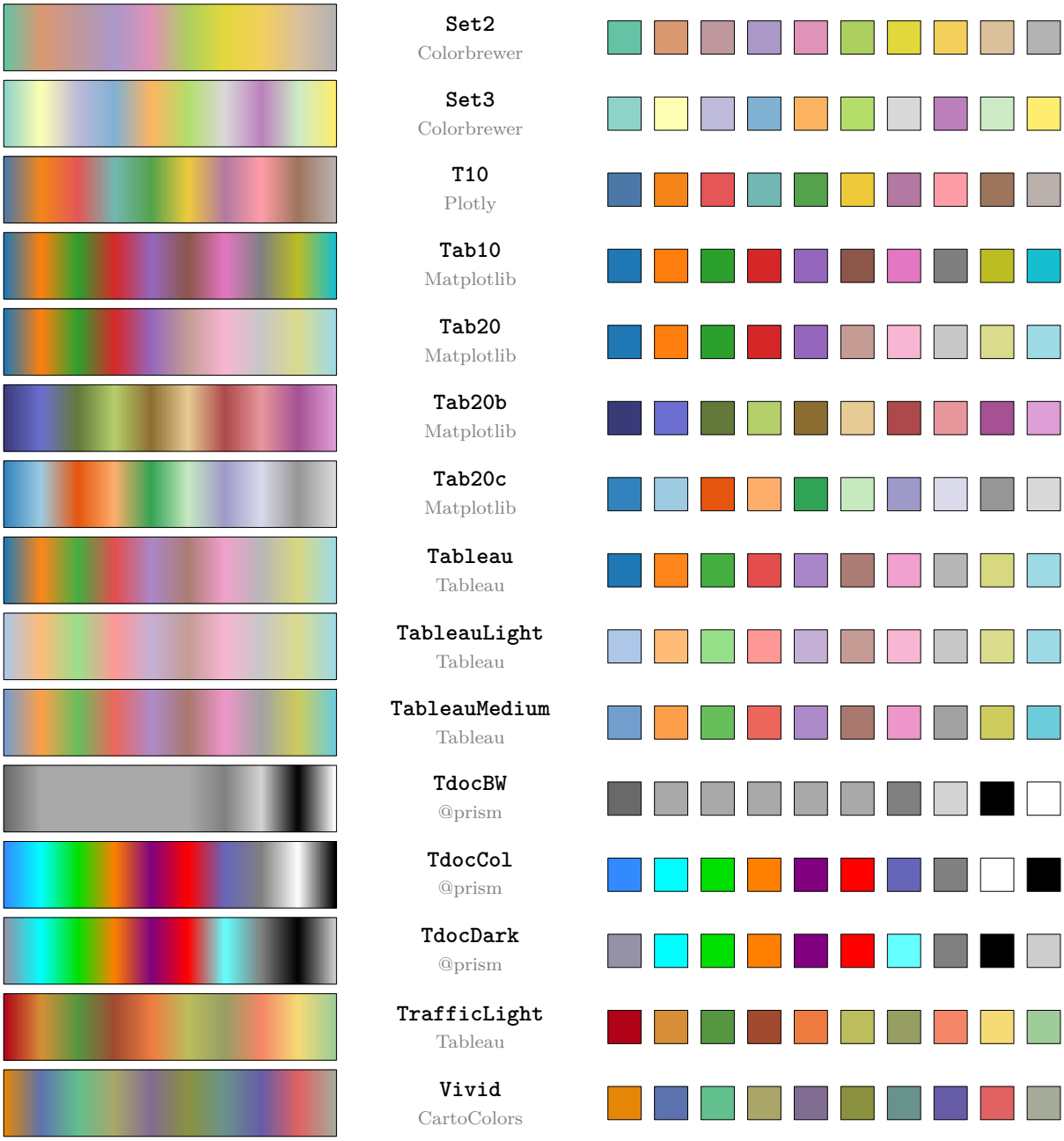
Colorblind-friendly palettes – 40 palettes



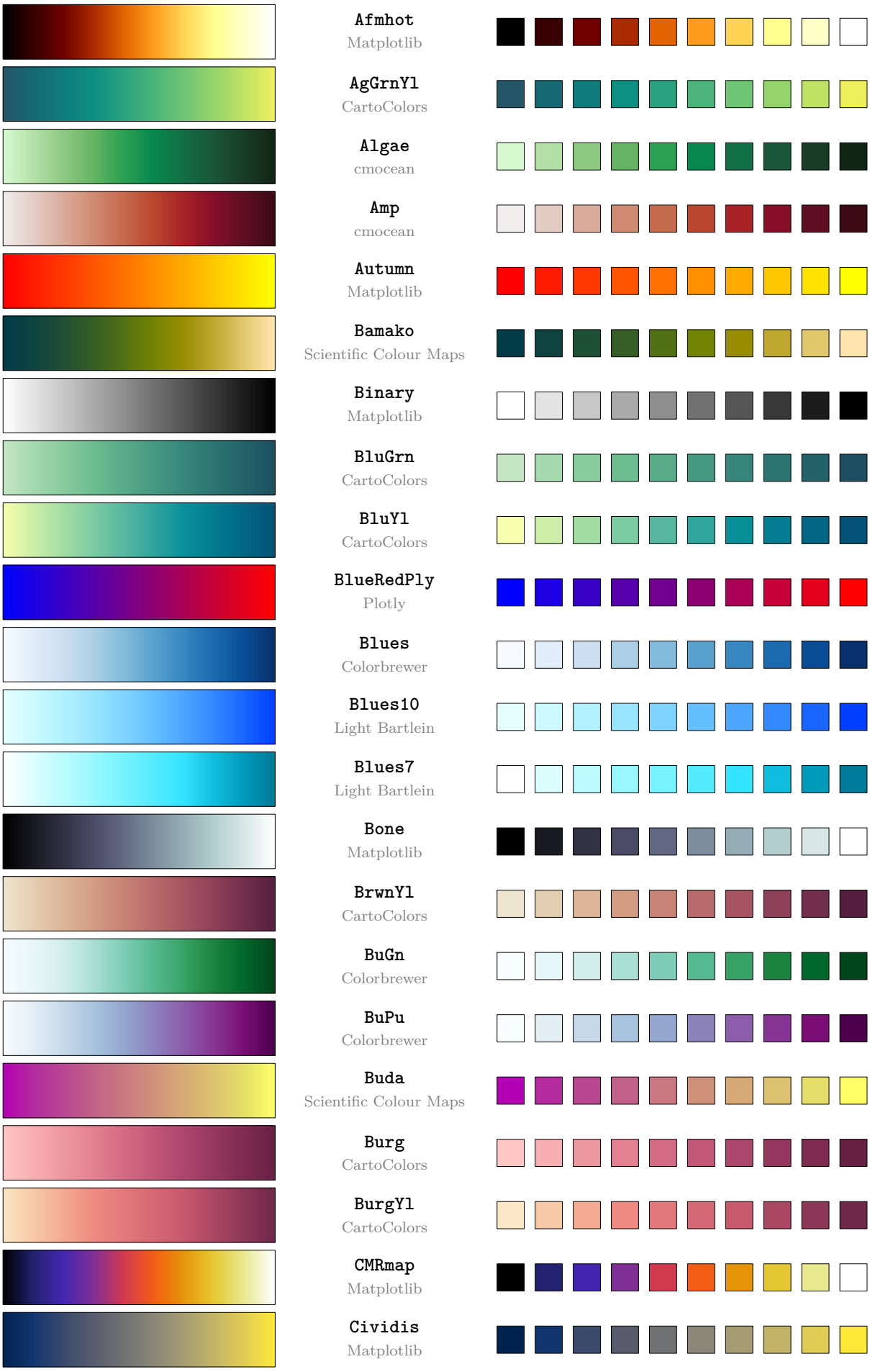
Glasgow
Scientific Colour Maps
GrayC
Scientific Colour Maps
Hawaii
Scientific Colour Maps
Imola
Scientific Colour Maps
Lajolla
Scientific Colour Maps
Lapaz
Scientific Colour Maps
Lipari
Scientific Colour Maps
Lisbon
Scientific Colour Maps
Managua
Scientific Colour Maps
Navia
Scientific Colour Maps
NaviaW
Scientific Colour Maps
Nuuk
Scientific Colour Maps
Oleron
Scientific Colour Maps
Oslo
Scientific Colour Maps
Roma
Scientific Colour Maps
RomaO
Scientific Colour Maps
Tofino
Scientific Colour Maps
Tokyo
Scientific Colour Maps
Turku
Scientific Colour Maps
Vanimo
Scientific Colour Maps
Vik
Scientific Colour Maps
VikO
Scientific Colour Maps

Semantic palettes – 37 palettes



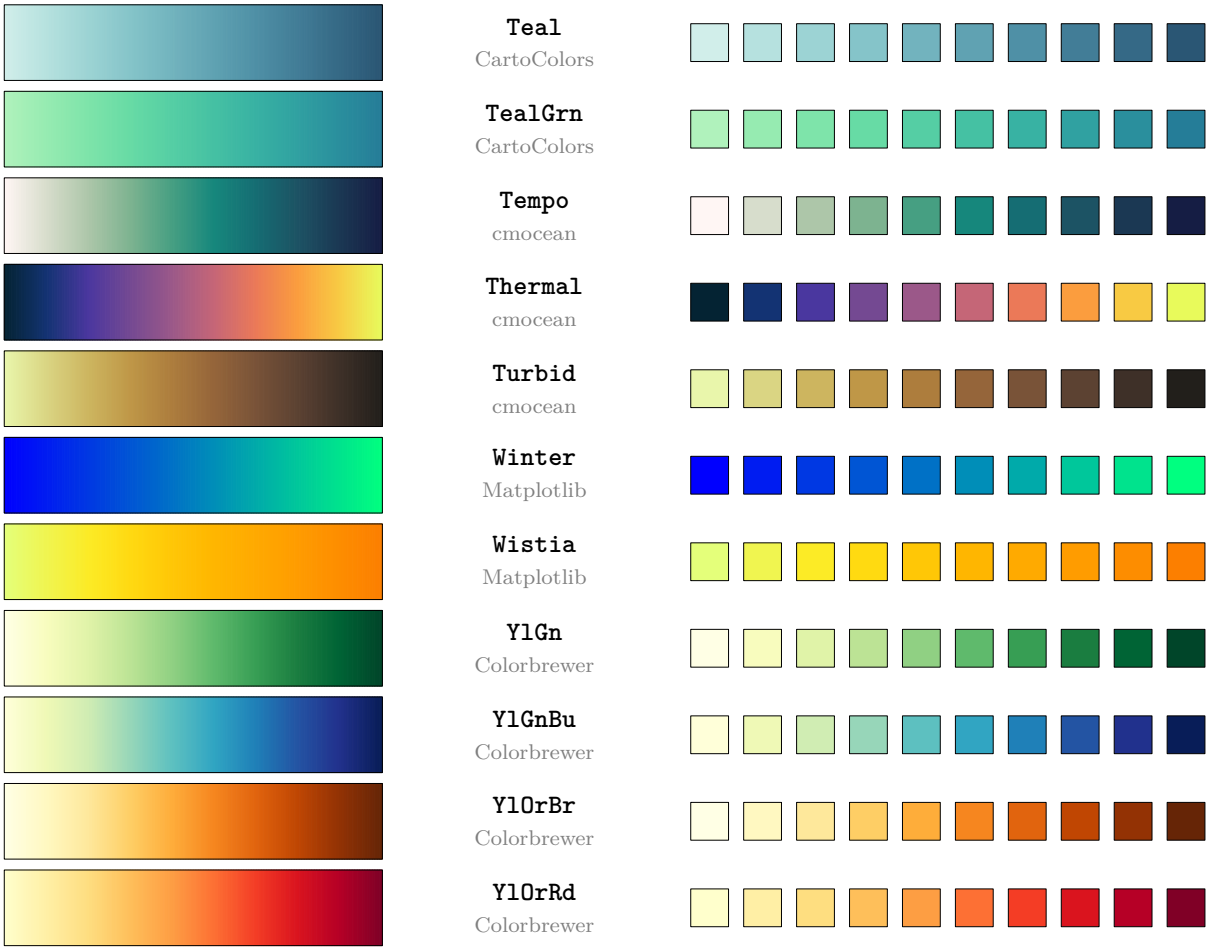

Set2
Colorbrewer
Set3
Colorbrewer
T10
Plotly
Tab10
Matplotlib
Tab20
Matplotlib
Tab20b
Matplotlib
Tab20c
Matplotlib
Tableau
Tableau
TableauLight
Tableau
TableauMedium
Tableau
TdocBW
@prism
TdocCol
@prism
TdocDark
@prism
TrafficLight
Tableau
Vivid
CartoColors

Two-color palettes – 79 palettes

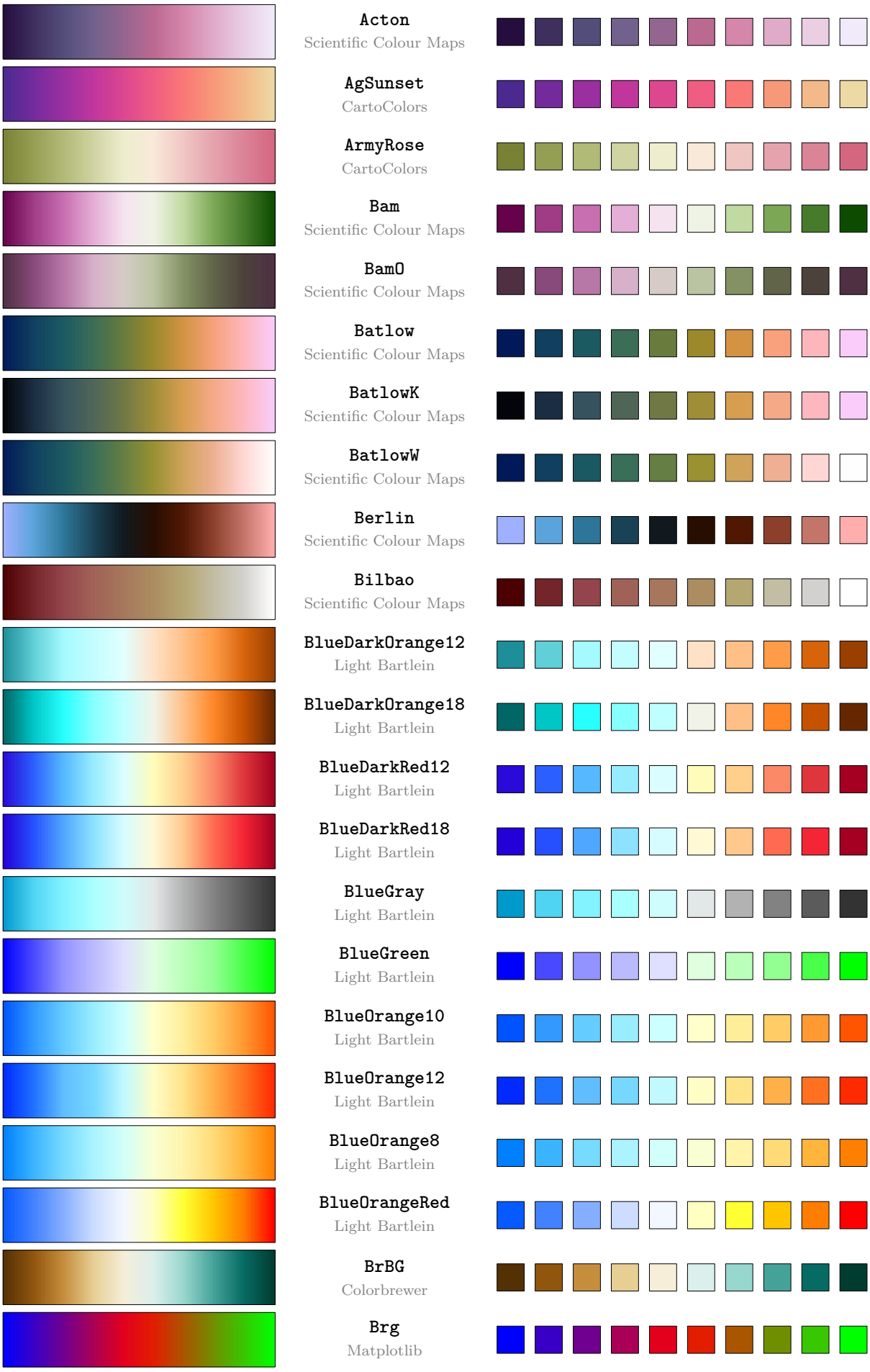


Copper
Matplotlib
DarkMint
CartoColors
Devon
Scientific Colour Maps
Electric
Plotly
Emrld
CartoColors
GistHeat
Matplotlib
GnBu
Colorbrewer
GrandBudapest2
Wes Anderson
GrayC
Scientific Colour Maps
GrayTab
Tableau
Grays
Matplotlib
Greens
Colorbrewer
Hot
Plotly
Ice
cmocean
Imola
Scientific Colour Maps
Inferno
Matplotlib
JimSpecial
Cubehelix
Lajolla
Scientific Colour Maps
Lapaz
Scientific Colour Maps
Magenta
CartoColors
Magma
Matplotlib
Matter
cmocean
Mint
CartoColors

OrRd
Colorbrewer
OrYel
CartoColors
Oranges
Colorbrewer
Oslo
Scientific Colour Maps
Peach
CartoColors
Pink
Matplotlib
PinkYl
CartoColors
Plasma
Matplotlib
PuBu
Colorbrewer
PuRd
Colorbrewer
Purp
CartoColors
PurpOr
CartoColors
Purple
Cubehelix
Purples
Colorbrewer
RdPu
Colorbrewer
Red
Cubehelix
RedOr
CartoColors
Reds
Colorbrewer
Solar
cmocean
Speed
cmocean
Spring
Matplotlib
Summer
Matplotlib
SunsetDark
CartoColors

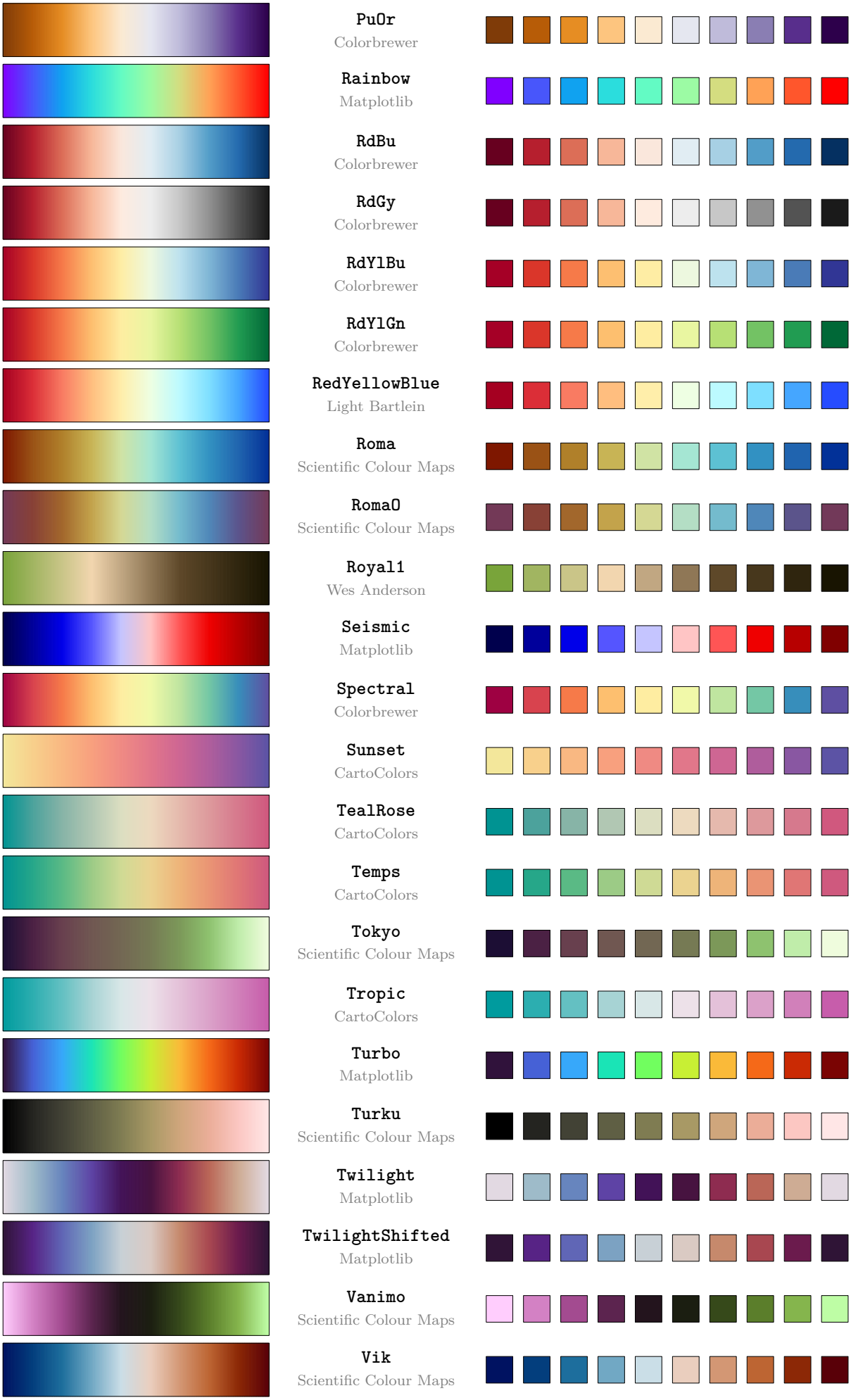

Teal
CartoColors
TealGrn
CartoColors
Tempo
cmocean
Thermal
cmocean
Turbid
cmocean
Winter
Matplotlib
Wistia
Matplotlib
YlGn
Colorbrewer
YlGnBu
Colorbrewer
YlOrBr
Colorbrewer
YlOrRd
Colorbrewer

Three-color palettes – 93 palettes



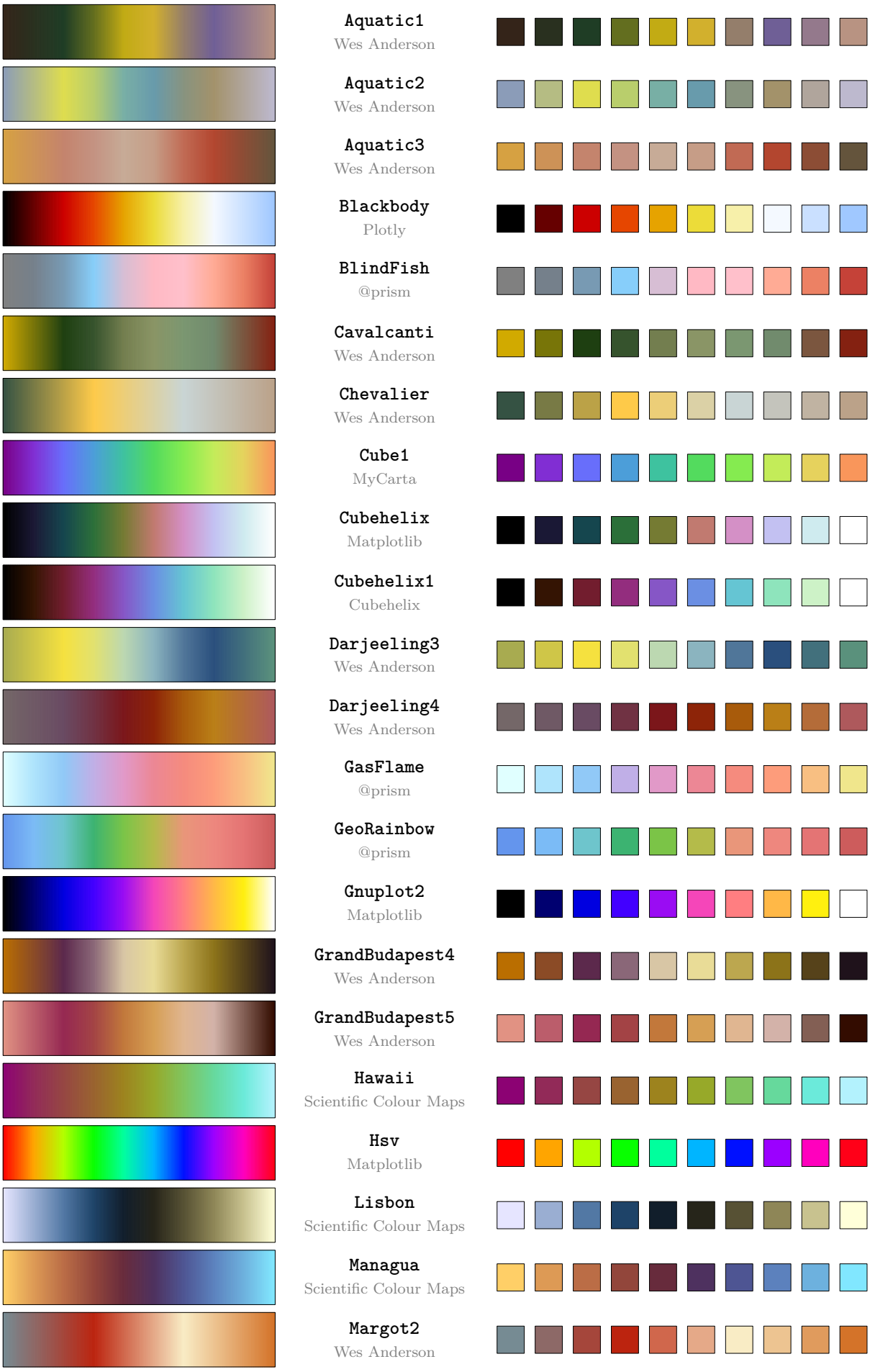
Broc
Scientific Colour Maps
BrocO
Scientific Colour Maps
BrownBlue10
Light Bartlein
BrownBlue12
Light Bartlein
BurningGrass
@prism
Bwr
Matplotlib
Cool
Matplotlib
Coolwarm
Matplotlib
Cork
Scientific Colour Maps
CorkO
Scientific Colour Maps
CubeYF
MyCarta
Cubehelix2
Cubehelix
Cubehelix3
Cubehelix
Curl
cmocean
Darjeeling1
Wes Anderson
Davos
Scientific Colour Maps
Deep
cmocean
Delta
cmocean
Dense
cmocean
Earth
CartoColors
Fall
CartoColors
FantasticFox1
Wes Anderson
Geyser
CartoColors

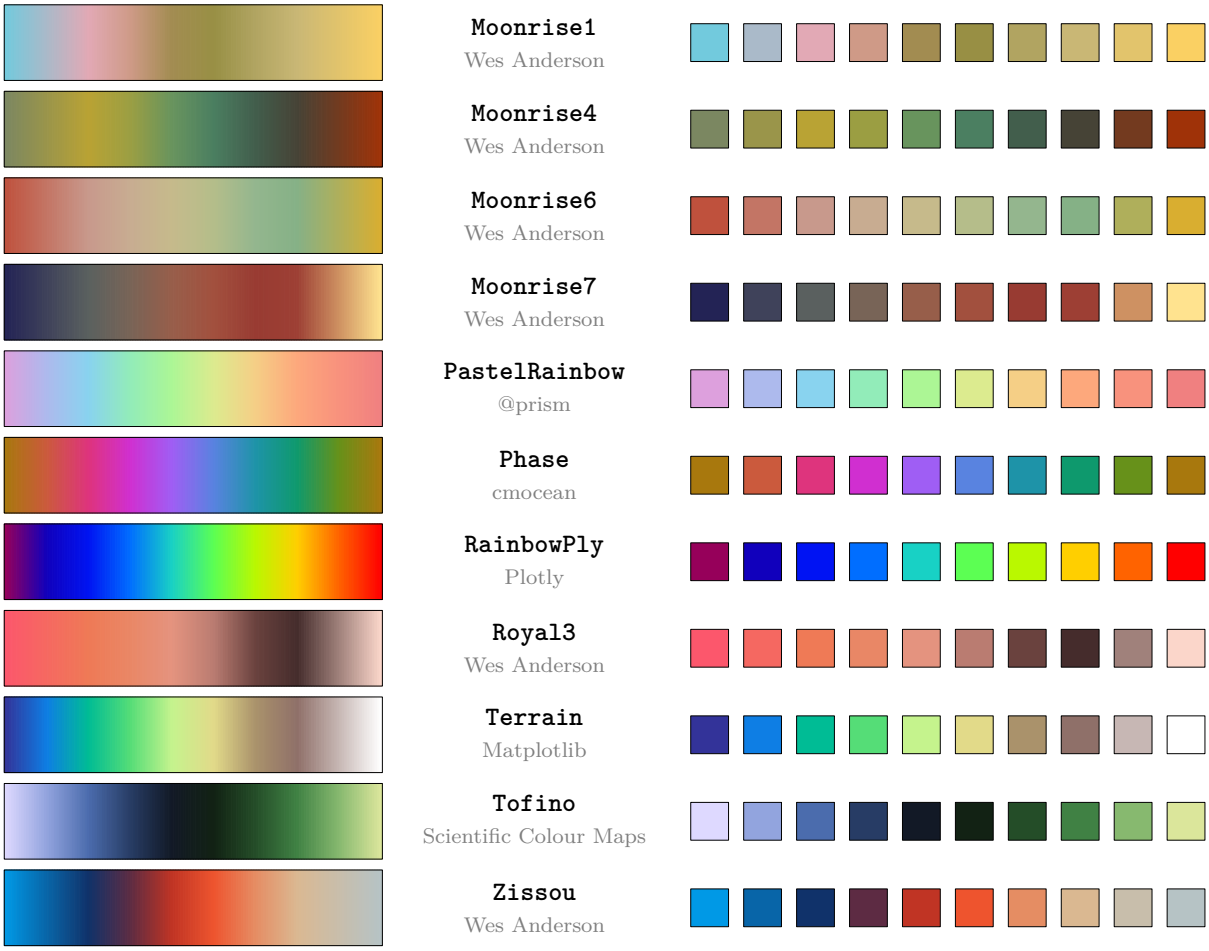
GistEarth
Matplotlib
Glasgow
Scientific Colour Maps
Gnuplot
Matplotlib
GrandBudapest1
Wes Anderson
GreenMagenta
Light Bartlein
Haline
cmocean
IsleOfDogs1
Wes Anderson
Jet
Plotly
Lemon
@prism
LinearL
MyCarta
Lipari
Scientific Colour Maps
Margot3
Wes Anderson
Moonrise3
Wes Anderson
Navia
Scientific Colour Maps
NaviaW
Scientific Colour Maps
Nuuk
Scientific Colour Maps
Ocean
Matplotlib
PRGn
Colorbrewer
PerceptualRainbow
Cubehelix
PiYG
Colorbrewer
Picnic
Plotly
Portland
Plotly
PuBuGn
Colorbrewer


PuOr
Colorbrewer
Rainbow
Matplotlib
RdBu
Colorbrewer
RdGy
Colorbrewer
RdYlBu
Colorbrewer
RdYlGn
Colorbrewer
RedYellowBlue
Light Bartlein
Roma
Scientific Colour Maps
RomaO
Scientific Colour Maps
Royal1
Wes Anderson
Seismic
Matplotlib
Spectral
Colorbrewer
Sunset
CartoColors
TealRose
CartoColors
Temps
CartoColors
Tokyo
Scientific Colour Maps
Tropic
CartoColors
Turbo
Matplotlib
Turku
Scientific Colour Maps
Twilight
Matplotlib
TwilightShifted
Matplotlib
Vanimo
Scientific Colour Maps
Vik
Scientific Colour Maps

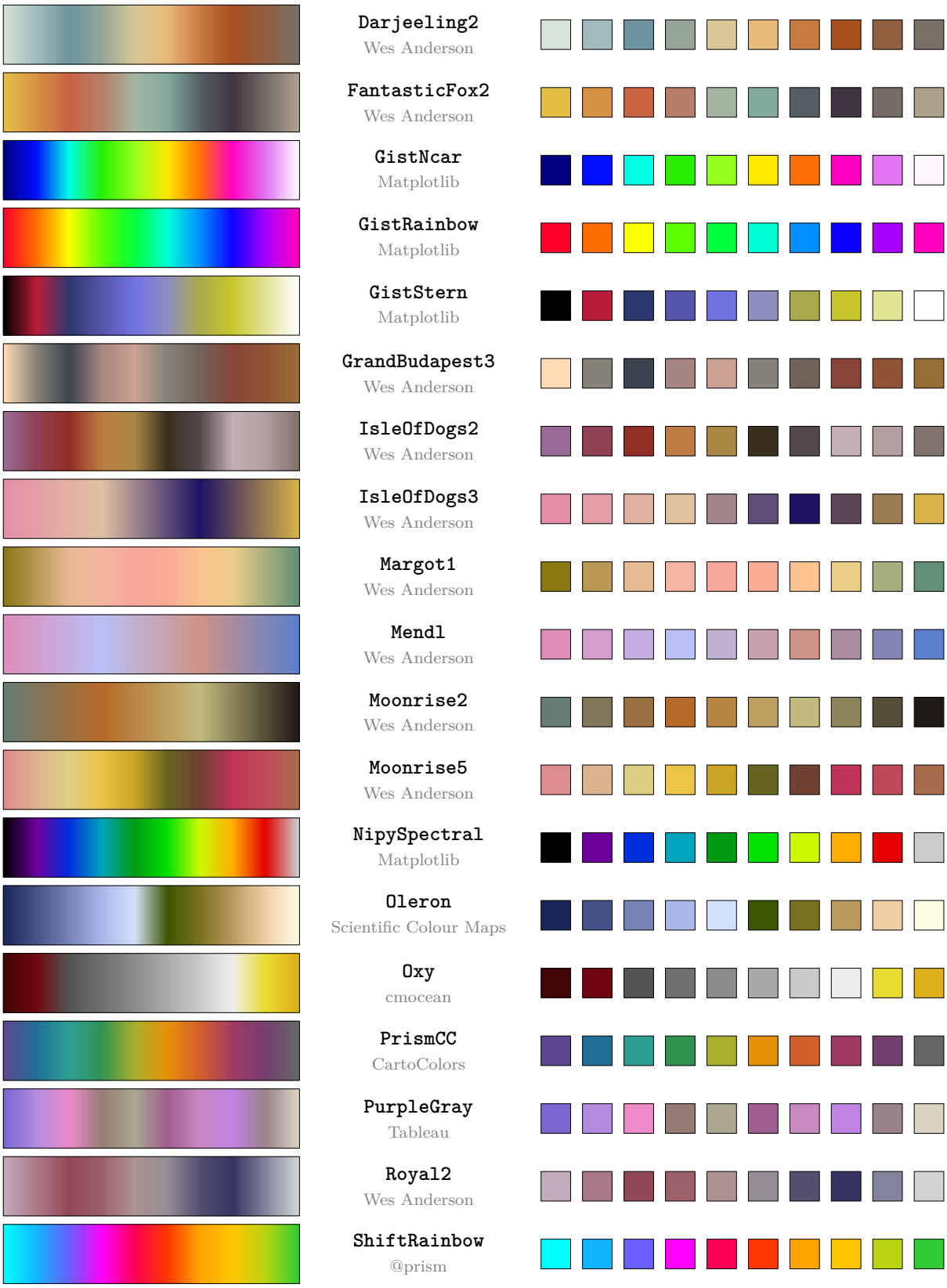
VikO
Scientific Colour Maps
Viridis
Matplotlib

Four-color palettes – 33 palettes




Moonrise1
Wes Anderson
Moonrise4
Wes Anderson
Moonrise6
Wes Anderson
Moonrise7
Wes Anderson
PastelRainbow
@prism
Phase
cmocean
RainbowPly
Plotly
Royal3
Wes Anderson
Terrain
Matplotlib
Tofino
Scientific Colour Maps
Zissou
Wes Anderson

More color palettes – 19 palettes



Appendix 2 – Similar palettes

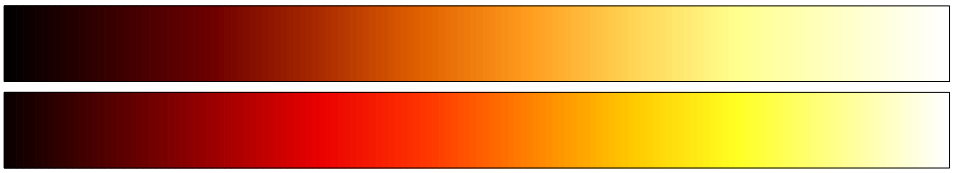
This appendix contains visually similar color palettes. While the differences between some are minimal, we have retained them to respect individual preferences.

Important.

Clusters were generated semi-automatically using a program that suggests similar palettes, followed by manual curation to retain only relevant groupings.[a] *This approach may occasionally miss some similarities. If you identify any omissions, please contact the author of* `@prism`.

[a] The palettes are analyzed in both light and dark modes.

Cluster #1



`Afmhot`

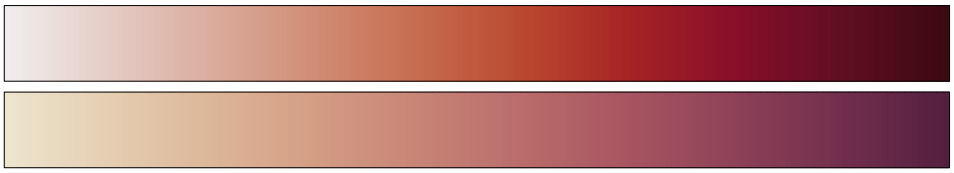
`Hot`

Cluster #2

`AgGrnYl`

`Summer`

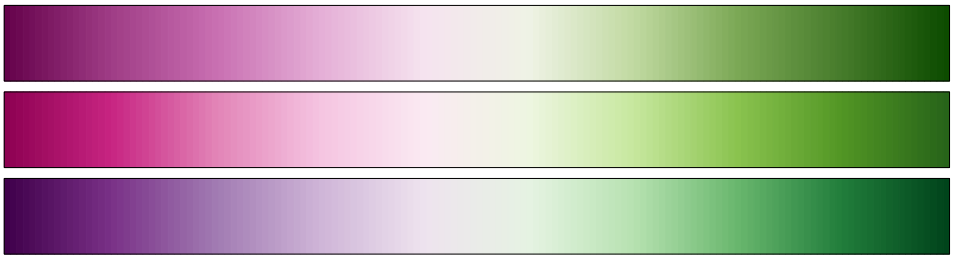
Cluster #3



`Amp`

`BrwnYl`

Cluster #4

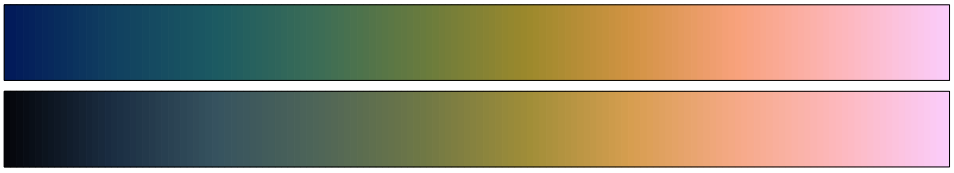


`Bam`

`PiYG`

`PRGn`

Cluster #5



`Batlow`

`BatlowK`

Cluster #6

Binary

Grays

Cluster #7

BluGrn

Emrld

Cluster #8

BlueDarkOrange12

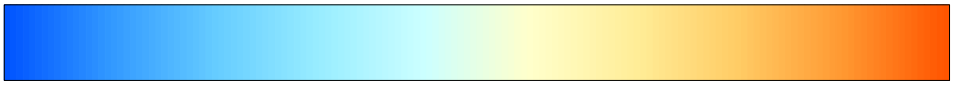
BlueDarkOrange18

Cluster #9

BlueDarkRed12

BlueDarkRed18

Cluster #10



BlueOrange10

BlueOrange8

Cluster #11

Blues

PuBu

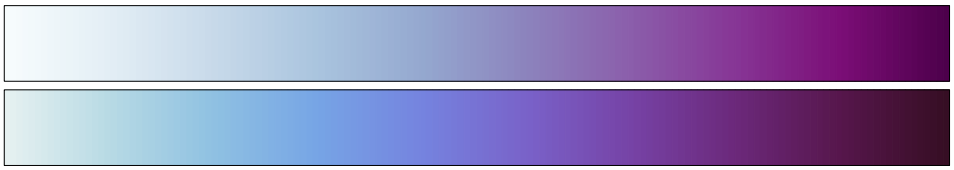
Cluster #12

Broc

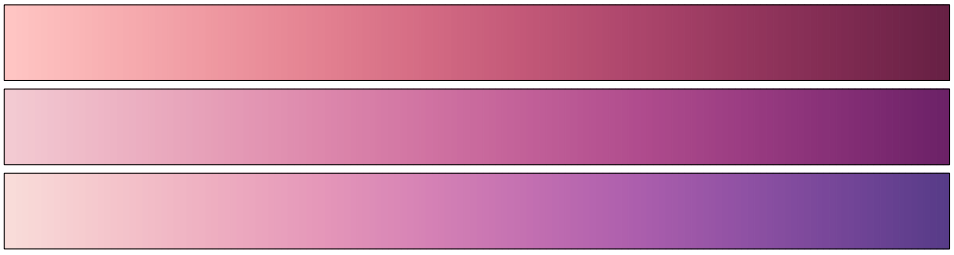
BrocO

Cluster #13

Cluster #14

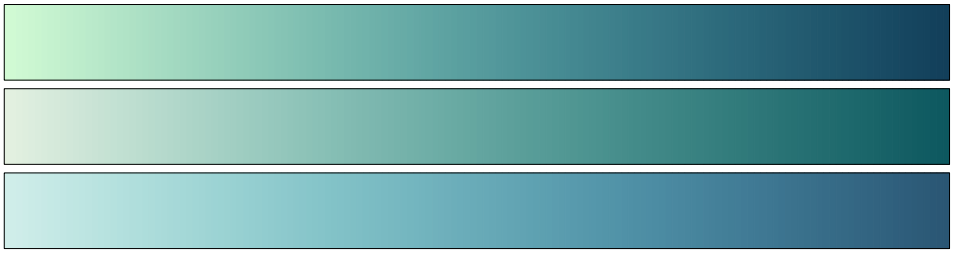


Cluster #15

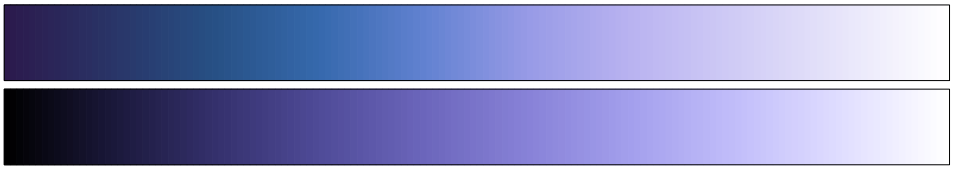


Cluster #16

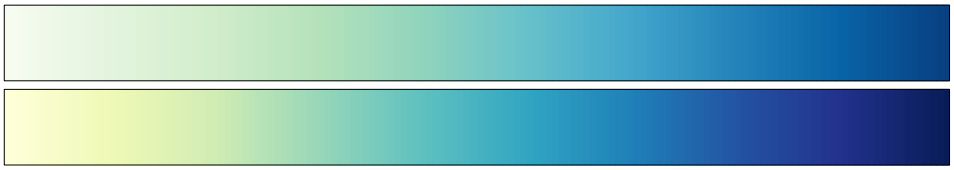
Cluster #17



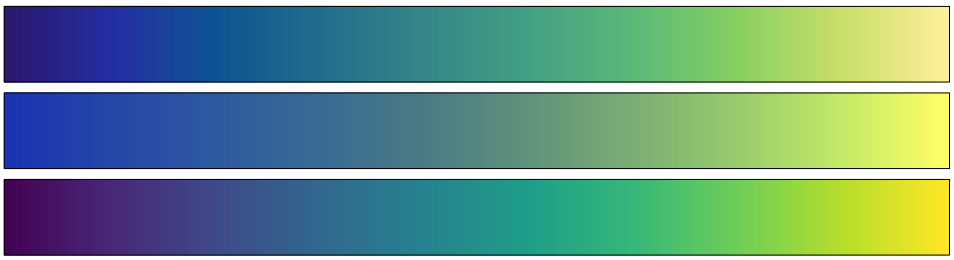
Cluster #18



Cluster #19



Cluster #20

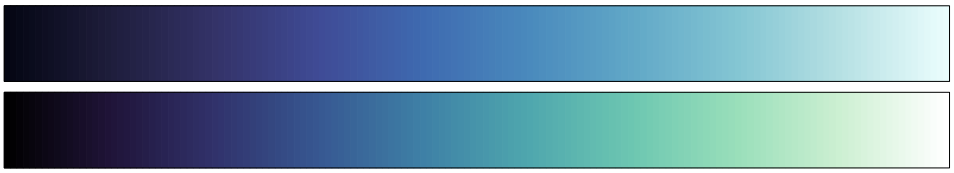


Haline

Imola

Viridis

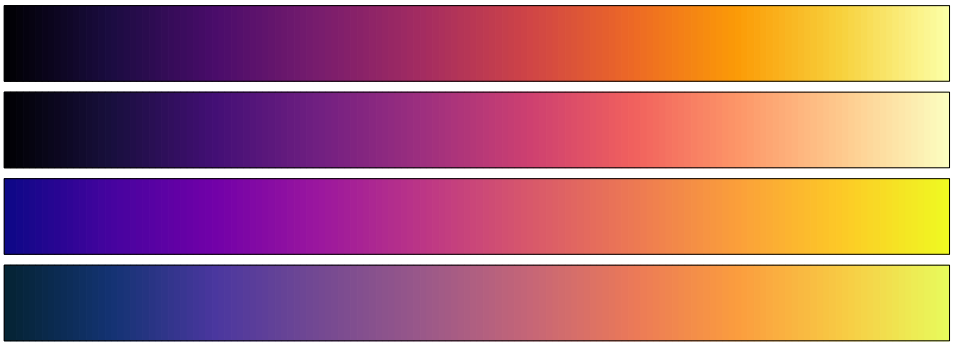
Cluster #21



Ice

JimSpecial

Cluster #22



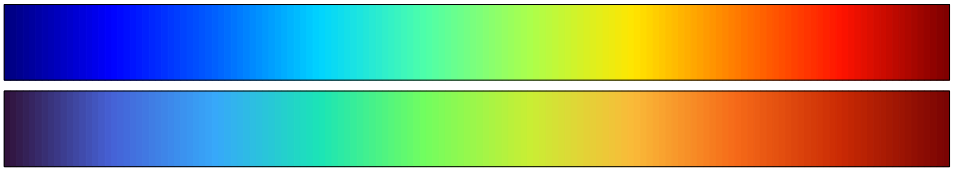
Inferno

Magma

Plasma

Thermal

Cluster #23



Jet

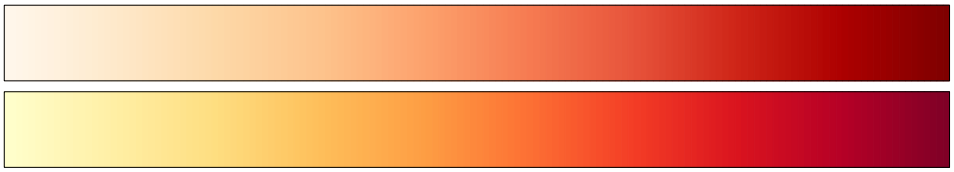
Turbo

Cluster #24

Navia

NaviaW

Cluster #25



OrRd

YlOrRd

Cluster #26

OrYel

Peach

Cluster #27

Oranges

YlOrBr

Cluster #28

RdYlBu

RedYellowBlue

Spectral

Cluster #29

Tab20

Tableau

Cluster #30

TealRose

Temps

Cluster #31

TwilightShifted

VikO